FloraSense AI

DETAILED DEEP LEARNING PROJECT REFERANCE REPORT

Plant Species Identification and Crop Disease Classification Using Transfer Learning

Example input images



Arjun.JPG: project input image



Mango.jpg: project input image

Project detail	Description
Primary framework	PyTorch + Torchvision
Main model	MobileNetV3-Large
Baseline model	ResNet18
Input size	224 × 224 RGB image
Recorded runtime	Python 3.13.1, CPU

Prepared from the supplied project description and reference report, including recorded metrics and model-efficiency results.

Contents

- Executive Summary
- 1. Introduction
- 2. Problem Statement and Scope
- 3. Objectives
- 4. System Architecture
- 5. Technology Stack
- 6. Dataset and Data Engineering
- 7. Image Preprocessing and Augmentation
- 8. Deep Learning Concepts
- 9. Training Workflow and Configuration
- 10. Evaluation Metrics
- 11. Measured Test Results
- 12. Efficiency and Deployment Comparison
- 13. Inference and Confidence-Aware Output
- 14. Plant-Care Knowledge Base
- 15. Benefits and Applications
- 16. Limitations and Risks
- 17. Recommended Improvements
- 18. Conclusion
- Appendix A. Project Artifact Index

Executive Summary

FloraSense AI is a computer-vision project that analyzes leaf images to identify supported plant species and classify crop-condition labels. It uses three independent classifiers: a 10-class Campus species model, a 12-class Kaggle species model, and a 38-class PlantVillage disease/healthy-condition model. The project applies transfer learning with MobileNetV3-Large and ResNet18, evaluates held-out test sets, and benchmarks model size and CPU inference speed.

The saved test results report accuracy from 99.065% to 99.767%. ResNet18 records the highest accuracy for all three tasks in the supplied final metrics, while MobileNetV3-Large is the recommended practical deployment model because it is around 16 MB and generally has lower CPU latency. These figures are dataset-specific and should not be interpreted as guaranteed real-world field accuracy.

- Separate label vocabularies are preserved instead of merging incompatible datasets.
- The training pipeline includes manifests, stratified splits, augmentation, validation-based checkpoint selection, and held-out testing.
- Inference returns top-k candidates and flags low top scores below the configured 0.40 threshold as uncertain.
- The next priority is independent field validation, confidence calibration, and prevention of potential data leakage.

1. Introduction

Plant identification is useful in education, campus biodiversity surveys, nurseries, gardens, and agricultural inspection. Visual recognition can be difficult because plant species may share similar leaf shapes and disease symptoms can overlap. FloraSense AI applies supervised image classification to provide an initial, AI-assisted interpretation of a leaf photograph.

The project is implemented as a local Jupyter Notebook. It demonstrates a complete deep-learning workflow: image discovery, data engineering, preprocessing, model training, validation, evaluation, checkpointing, efficiency benchmarking, and single-image inference.

2. Problem Statement and Scope

Manual plant identification often depends on botanical knowledge. Crop disease recognition can also be challenging when symptoms are subtle, early-stage, or visually similar. The project aims to classify supported plant images using convolutional neural networks and transfer learning.

Input	Expected output
Leaf or plant image	Top-k supported species predictions
Image from a supported crop	Candidate disease or healthy-condition class
Model score	Confidence-aware status and uncertainty warning
Recognized class	Verified care profile, only if one is available

The scope is limited to the labels present in each training dataset. A model cannot reliably identify an arbitrary species outside its supported classes. Disease outputs are candidate visual classifications, not confirmed agricultural diagnoses.

3. Objectives

1. Build a leaf-image classification pipeline for supported plant species.
2. Train separate models for Campus, Kaggle species, and PlantVillage crop-condition taxonomies.

3. Use transfer learning to reduce training requirements and compare MobileNetV3-Large with ResNet18.
4. Evaluate models using accuracy, precision, recall, F1-score, confusion matrices, model size, latency, and FPS.
5. Provide top-k predictions and flag low-scoring predictions as uncertain.
6. Preserve class mappings, checkpoints, manifests, and environment information for reproducibility.
7. Return plant-care information only from a manually reviewed knowledge base.

4. System Architecture

Image decoding and preprocessing are shared. The classification models remain separate because each dataset uses a different label vocabulary. This separation prevents class indices from being mixed and keeps outputs tied to the dataset on which the model was trained.

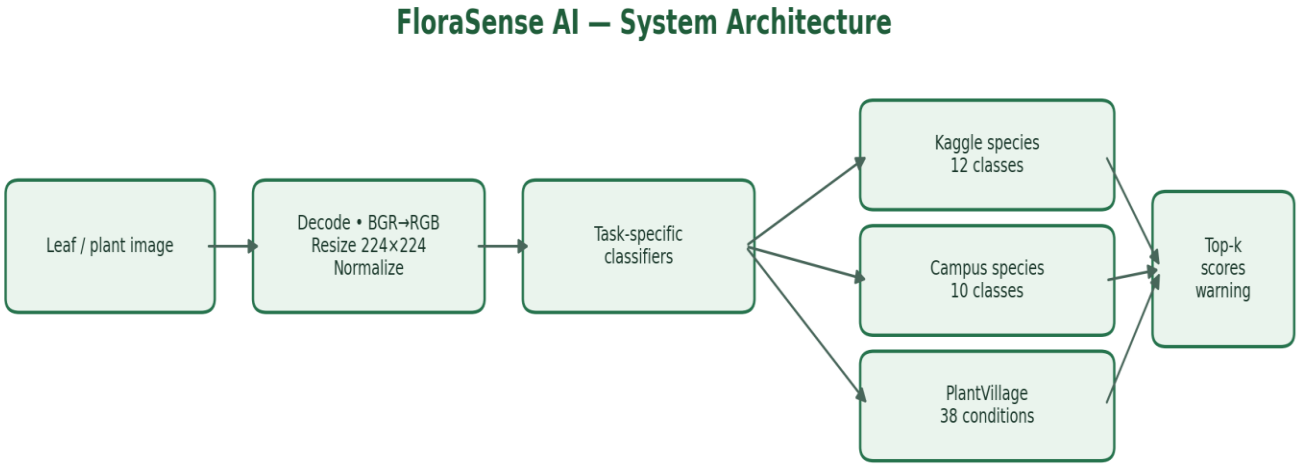


Figure 1. End-to-end FloraSense AI architecture. Each task-specific model has its own class vocabulary.

Task	Dataset	Classes	Purpose
Campus species	Campus plant dataset	10	Common campus and garden plant recognition
Kaggle species	Kaggle species dataset	12	Indian plant species identification
PlantVillage disease	PlantVillage	38	Crop disease and healthy-condition classification

The notebook can run all three models on the same image, but their outputs must be interpreted independently. A disease prediction should only be considered relevant when the input crop is compatible with the disease model's label.

5. Technology Stack

Technology	Role in the project
Python 3.13.1	Main programming language
Jupyter Notebook / VS Code	Interactive development and experimentation
PyTorch 2.14.1+cpu	Model training and inference in the recorded environment
Torchvision 0.29.1+cpu	Pretrained architectures and image transforms
MobileNetV3-Large	Main lightweight model
ResNet18	Baseline comparison model
OpenCV 5.0.0	Image decoding, BGR-to-RGB conversion, resizing
Pillow	Image object conversion

NumPy / Pandas	Numerical operations, manifests, metrics tables
Scikit-learn	Classification metrics, reports, confusion matrices
Matplotlib / tqdm	Plots and training progress display

The recorded environment uses CPU execution; CUDA is not used in the supplied configuration. Package versions are reproduced from the reference material and should be verified against the actual runtime before attempting to recreate the environment.

6. Dataset and Data Engineering

6.1 Dataset sources and label spaces

Three local datasets are used. The Campus dataset contains 10 plant classes, the Kaggle species dataset contains 12 classes, and PlantVillage contains 38 crop-condition classes. The PlantVillage label space includes disease categories for crops such as apple, tomato, potato, corn, grape, and pepper, as well as healthy categories.

6.2 Image manifests

The notebook creates CSV manifests that record image path, dataset name, folder label, species label, health label, disease label, and split name. Important manifest artifacts include campus_species_manifest.csv, kaggle_species_manifest.csv, and plantvillage_disease_manifest.csv. Manifests make data selection auditable and support reproducible evaluation.

6.3 Dataset splitting

For datasets without predefined splits, the project uses stratified train/validation/test splitting; the reference report describes a 70:15:15 split. Existing TRAIN, VALID, and TEST folders for the Campus dataset are preserved. Stratification attempts to keep class proportions similar across splits.

6.4 Leakage risk

If several images show the same physical leaf or plant, a random image-level split may place highly similar examples in both training and test sets. That can inflate test performance. A stronger production evaluation should split by plant identity, leaf identity, collection location, or capture session.

6.5 Data quality controls

- Discover image files using supported extensions.
- Reject unreadable images during OpenCV validation.
- Generate deterministic class-to-index mappings.
- Keep train, validation, and test roles separate.
- Record the split in the manifest so each reported metric can be traced to its evaluation data.

7. Image Preprocessing and Augmentation

OpenCV loads images in BGR channel order. The notebook converts each image to RGB, resizes it to 224 × 224 pixels, and converts it into the tensor representation expected by the network.

Stage	Processing
Decode	Read the image and reject unreadable files
Color conversion	BGR → RGB
Resize	224 × 224 pixels

Training augmentation	Random rotation, color jitter, horizontal flip
Tensor conversion	Convert image pixels into a tensor
Normalization	ImageNet mean and standard deviation
Validation/test	Deterministic transform; no random augmentation

Augmentation is applied only to training images. It introduces moderate variations in orientation, brightness, and color to reduce sensitivity to capture conditions. Validation and test transforms remain deterministic so that evaluation is repeatable.

8. Deep Learning Concepts

8.1 Supervised multi-class classification

Each training image is paired with a known class label. The network produces a vector of logits; the class with the highest logit is selected as the top prediction. Each model predicts from its own fixed label vocabulary.

8.2 Convolutional neural networks

CNN layers learn visual features hierarchically. Early layers can respond to edges, lines, and color boundaries. Intermediate layers can represent veins, spots, margins, and textures. Deeper layers combine these features into evidence that distinguishes plant species or disease-condition classes.

8.3 Transfer learning

The project starts from ImageNet-pretrained MobileNetV3-Large and ResNet18 networks. The final classification layer is replaced with a layer whose output size matches the number of classes for the current task. Pretrained visual features provide a useful starting point instead of learning every feature from scratch.

8.4 MobileNetV3-Large

MobileNetV3-Large is the main deployment candidate. Its efficient blocks, depthwise-style convolution operations, inverted residual structures, and channel-attention mechanisms are designed to reduce compute and memory requirements. The recorded checkpoint sizes are about 16 MB.

8.5 ResNet18

ResNet18 is the comparison baseline. Residual connections add an input representation to a learned transformation, commonly written as $y = F(x) + x$. This helps optimization and makes ResNet18 a reliable benchmark, though its recorded checkpoints are around 43 MB.

8.6 Loss and optimization

Cross-entropy loss penalizes a model when it assigns low probability to the correct class. Adam updates model parameters using adaptive estimates of gradient moments. Weight decay adds regularization and can help reduce overfitting.

$$\text{Cross-entropy: } L = -\Sigma_i y_i \log(p_i)$$

8.7 Learning-rate scheduling and early stopping

The learning rate is reduced when validation macro-F1 stops improving. The best checkpoint is selected using validation macro-F1, and early stopping terminates training if this metric does not improve for four epochs. The test set is not used to choose the checkpoint.

9. Training Workflow and Configuration

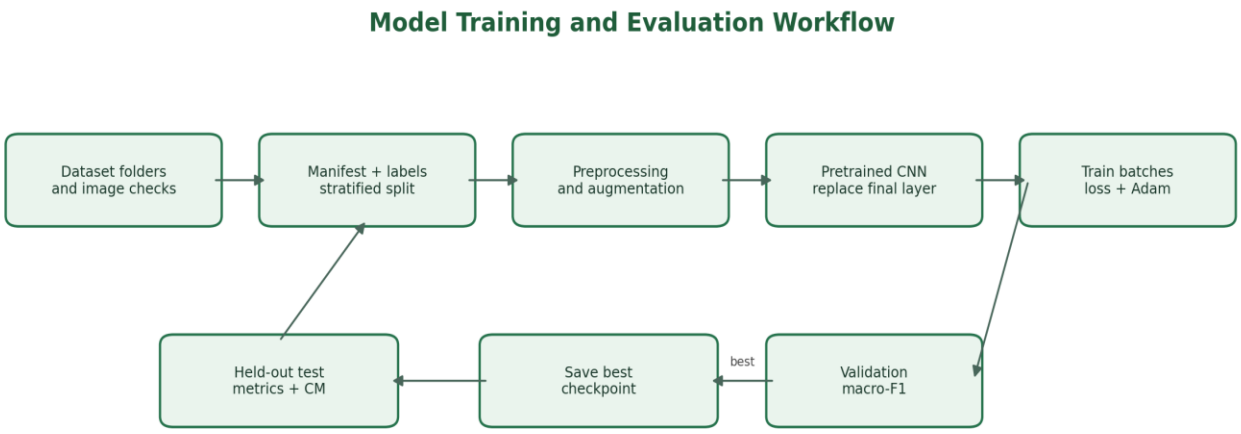


Figure 2. Training and evaluation workflow. Test data is held out until final evaluation.

Parameter	Recorded value
Input image size	224 × 224
Batch size	32
Learning rate	0.0001
Weight decay	0.0001
Maximum epochs	10
Early-stopping patience	4 epochs
Random seed	42
Execution device	CPU

9.1 Core training sequence

8. Create dataset manifests and split records.
9. Build Dataset and DataLoader objects.
10. Load an ImageNet-pretrained architecture and replace its final layer.
11. Run a forward pass to produce logits.
12. Calculate cross-entropy loss against the labels.
13. Run backpropagation and update parameters with Adam.
14. Evaluate on the validation split without updating weights.
15. Save the checkpoint with the best validation macro-F1.
16. Evaluate the selected checkpoint on the held-out test split.

9.2 Checkpoint contents

Each checkpoint stores the architecture name, task name, model weights, class names, class-to-index mapping, image size, normalization statistics, best validation macro-F1, and random seed. This metadata is necessary to reconstruct the same inference pipeline and correctly translate output indices into labels.

10. Evaluation Metrics

Metric	Formula / definition	Interpretation
Accuracy	Correct predictions ÷ total predictions	Overall correctness
Precision	TP ÷ (TP + FP)	How often a predicted class is correct
Recall	TP ÷ (TP + FN)	How many actual examples of a class are found
F1-score	2 × Precision × Recall ÷ (Precision + Recall)	Balances precision and recall
Macro average	Unweighted mean of class-level metric values	Each class has equal influence
Weighted average	Class-level metrics weighted by class support	Large classes contribute more

For multi-class classification, precision, recall, and F1 are commonly calculated one class at a time and then macro-averaged. Macro-F1 is useful when class counts differ because every class contributes equally to the final average.

10.1 Confusion matrix

A confusion matrix compares true labels with predicted labels. Correct classifications appear on the main diagonal; off-diagonal values identify pairs of classes that the model confuses. Confusion matrices should be reviewed alongside aggregate metrics to find weaknesses hidden by overall accuracy.

10.2 Operational metrics

The project also measures checkpoint size, batch-one forward-pass latency, and frames per second (FPS). The benchmark starts with an already preprocessed 224 × 224 tensor, so it excludes image loading and OpenCV preprocessing. End-to-end application latency will therefore be higher.

11. Measured Test Results

The following values are taken from the supplied final_test_metrics.csv summary. They describe the saved held-out test sets and are not an independent real-world validation.

Task	Model	Test images	Accuracy	Macro P	Macro R	Macro F1	Size
Kaggle species	MobileNetV3-Large	642	99.065%	99.088%	98.926%	98.993%	16.293 MB
Kaggle species	ResNet18	642	99.377%	99.503%	99.115%	99.296%	42.736 MB
PlantVillage disease	MobileNetV3-Large	8,146	99.619%	99.451%	99.365%	99.405%	16.426 MB
PlantVillage disease	ResNet18	8,146	99.767%	99.629%	99.731%	99.678%	42.789 MB
Campus species	MobileNetV3-Large	3,634	99.725%	99.722%	99.735%	99.728%	16.283 MB

Campus species	ResNet18	3,634	99.752%	99.762%	99.762%	99.762%	42.732 MB

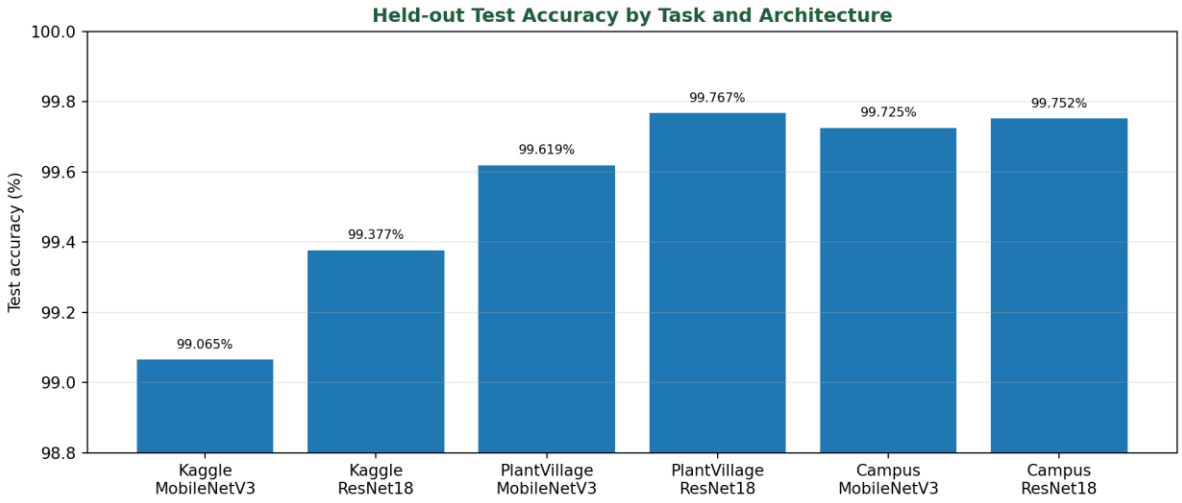


Figure 3. Held-out test accuracy by task and architecture, as recorded in the supplied final metrics.

ResNet18 has the highest recorded accuracy in each task: 99.377% for Kaggle species, 99.767% for PlantVillage disease classification, and 99.752% for Campus species. MobileNetV3-Large remains close in accuracy while using substantially smaller checkpoints.

11.1 Per-class observations

- Campus MobileNetV3-Large: most listed classes have F1 near 0.99–1.00; Tulsi has the lowest reported F1 at approximately 0.98.
- Kaggle MobileNetV3-Large: Bael, Basil, Chinar, Jatropha, and Lemon have reported F1 values of 1.00; Alstonia Scholaris is approximately 0.98.
- PlantVillage MobileNetV3-Large: Corn Cercospora leaf spot has F1 0.95 and recall 0.92; Corn Northern Leaf Blight has F1 0.97. Tomato Early Blight, Spider Mites, and Target Spot are around F1 0.98.

The more difficult classes should receive targeted review and real-world testing. Very high scores can also occur when the test images are clean, highly similar to training images, or affected by split leakage; the report therefore treats these as recorded test results, not a guarantee of field performance.

12. Efficiency and Deployment Comparison

Task	Model	Latency (ms)	FPS	Model size
Kaggle species	MobileNetV3-Large	13.424	74.49	16.29 MB
Kaggle species	ResNet18	17.555	56.96	42.74 MB
PlantVillage disease	MobileNetV3-Large	11.534	86.70	16.43 MB
PlantVillage disease	ResNet18	69.380	14.41	42.79 MB
Campus species	MobileNetV3-Large	12.218	81.84	16.28 MB
Campus species	ResNet18	27.883	35.86	42.73 MB

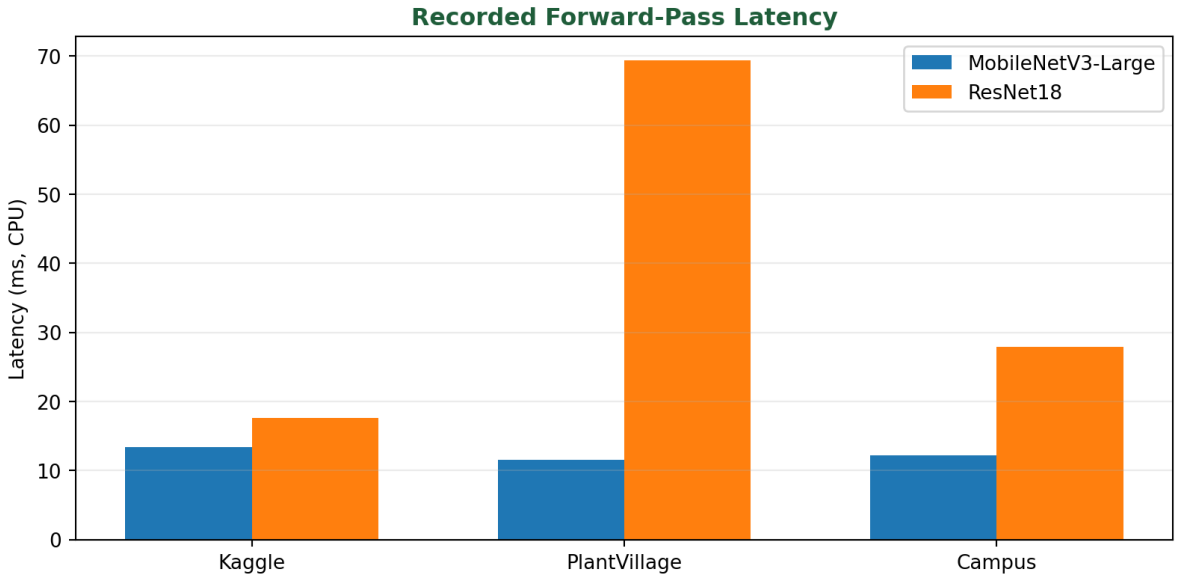


Figure 4. Recorded CPU forward-pass latency. Lower latency is better.

MobileNetV3-Large is approximately 16 MB per checkpoint and has lower recorded latency across all three tasks. ResNet18 is approximately 43 MB and has a small accuracy advantage on the current test sets. The practical choice depends on whether deployment efficiency or maximum benchmark accuracy is the priority.

Requirement	Recommended choice
Mobile / edge deployment	MobileNetV3-Large
Low memory and fast CPU inference	MobileNetV3-Large
Maximum recorded benchmark accuracy	ResNet18
Research baseline comparison	ResNet18

13. Inference and Confidence-Aware Output

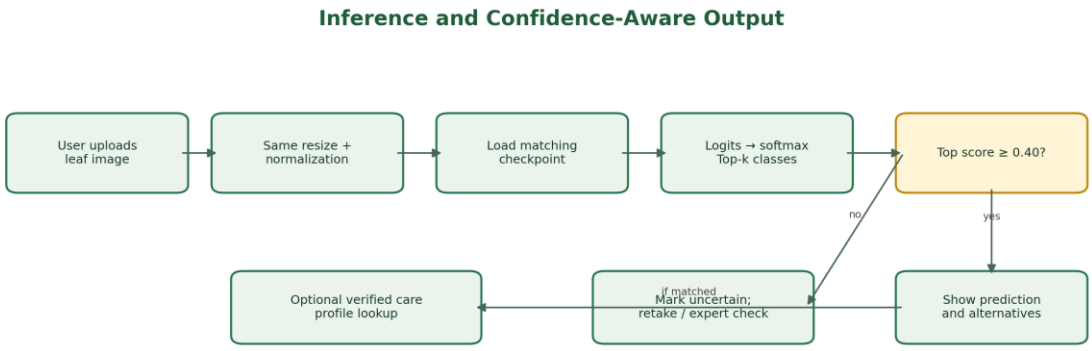


Figure 5. Single-image inference, confidence thresholding, and optional care-profile lookup.

At inference time, the model is set to evaluation mode. The image follows the same RGB conversion, resize, tensor conversion, and normalization path used during evaluation. Logits are converted to softmax scores, and the top-k candidate classes are returned.

The notebook uses a default threshold of 0.40. If the top score is below this threshold, the system reports an uncertain result and recommends capturing a clearer image or seeking expert confirmation. Softmax scores are not automatically calibrated probabilities; a score of 0.95 does not necessarily mean a true 95% chance of correctness.

13.1 Recommended result structure

Field	Example / rule
Species	Mango (example only)
Score	0.96 (illustrative example)
Alternatives	Guava, Neem (illustrative)
Status	supported_prediction if the class is supported and threshold is met
Disease	Only when crop compatibility is confirmed
Warning	AI-assisted prediction; confirm important decisions with an expert

13.2 Combined analysis safety

Running the species models and disease model on the same image does not automatically make their outputs mutually compatible. A PlantVillage disease candidate should be shown only when the predicted crop is compatible with that disease label. Otherwise, the output should be marked unsupported or withheld.

13.3 Confidence calibration

- Tune thresholds on validation data, not on the test set.
- Use temperature scaling and reliability diagrams.
- Measure Expected Calibration Error (ECE).
- Consider class-specific thresholds and out-of-distribution detection.

14. Plant-Care Knowledge Base

The optional care dictionary contains manually entered, reviewed profiles. A profile may include common name, scientific name, description, soil, sunlight, watering, temperature, common problems, management notes, source name, source URL, and review date.

If a class has no verified profile, the system should return no care profile rather than inventing advice. In particular, it should not generate unsupported pesticide dosage, chemical treatment, medical claims, exact soil requirements, or disease-treatment instructions. Agricultural guidance should be checked against appropriate local expert or extension sources.

15. Benefits and Applications

- Agriculture: preliminary crop-disease screening and field inspection assistance.
- Campus and gardens: plant inventory, biodiversity mapping, and botany education.
- Nurseries: plant recognition and customer-support workflows.
- Environmental monitoring: local species cataloging and green-space surveys.
- Mobile and edge AI: smaller MobileNetV3-Large checkpoints support offline inference experiments.
- Education and research: demonstrates dataset engineering, CNNs, transfer learning, evaluation, and reproducible experimentation.

16. Limitations and Risks

16.1 Dataset bias and domain shift

PlantVillage images are often relatively clean and centered. Real field images may include shadows, complex backgrounds, blur, insects, occlusion, multiple leaves, poor lighting, different cameras, cultivars, and disease stages. Performance may decrease when the input distribution differs from the training data.

16.2 Supported classes only

The classifier only recognizes the classes used in training. An unsupported plant may still receive a high softmax score for one of the known classes; the current threshold alone does not guarantee reliable open-set rejection.

16.3 Disease predictions are not diagnoses

Visual classification is only one source of evidence. Confirmation may require expert inspection, multiple photographs, field conditions, weather and soil context, symptom progression, or laboratory analysis.

16.4 Potential data leakage

If images of the same leaf or plant are distributed across training and testing, test results may be optimistic. Group-based splitting is recommended.

16.5 Inconsistent recorded metric files

The source material notes differences between some values in all_model_metrics.csv and final_test_metrics.csv, including a Kaggle ResNet18 result. Before publication, rerun evaluation with the exact saved checkpoints, one fixed manifest, verified class mappings, and recorded checkpoint hashes.

17. Recommended Improvements

17. Collect and label real field images from different phones, lighting conditions, locations, cultivars, and disease stages.
18. Adopt group-based train/validation/test splits by plant, leaf, location, or capture session.
19. Add a simple frontend with Streamlit or Gradio; for a larger application, consider React with a FastAPI backend.
20. Use leaf detection or segmentation (for example, YOLO, Mask R-CNN, or U-Net) when images contain multiple leaves or complex backgrounds.
21. Improve uncertainty handling with calibration, entropy or energy-based detection, and open-set recognition.
22. Add Grad-CAM visualizations to check whether the network focuses on leaf regions rather than background cues.
23. Create automated tests for unreadable images, class mappings, checkpoint loading, preprocessing consistency, single-image inference, and output schema.
24. Evaluate deployment formats such as TorchScript, ONNX, quantized PyTorch, or mobile conversion after validating model parity.
25. Resolve the metric-file discrepancy and publish one reproducible evaluation report.

18. Conclusion

FloraSense AI demonstrates an end-to-end deep-learning workflow for plant species identification and crop-condition classification. It combines data manifests, preprocessing, augmentation, ImageNet transfer learning, model comparison, validation-based checkpointing, held-out testing, performance benchmarking, and confidence-aware inference.

The supplied test metrics are very strong for the current datasets. ResNet18 achieves slightly higher recorded accuracy, while MobileNetV3-Large offers a more practical balance of accuracy, checkpoint size, and CPU speed. MobileNetV3-Large is therefore the recommended starting point for a resource-constrained prototype.

The most important next step is independent validation on real-world field images. A deployment-ready system should include calibrated uncertainty, crop-compatible disease logic, verified care content, and clear wording that its output is decision support rather than a definitive diagnosis.

Appendix A. Project Artifact Index

Artifact	Purpose
FloraSense_Final_Input_Based_DL.ipynb	Training, evaluation, inference, and benchmarking workflow
models/*.pth	Saved best model checkpoints
outputs/*_manifest.csv	Dataset paths, labels, and split records
outputs/*_class_mapping.json	Class-index lookup for inference
outputs/final_test_metrics.csv	Held-out test metrics
outputs/model_efficiency_metrics.csv	Latency, FPS, and checkpoint sizes
outputs/*_classification_report.txt	Per-class precision, recall, and F1 reports
environment.json	Recorded environment and configuration metadata

This report is based on the supplied FloraSense AI project description and reference PDF. Where a value is described as an example, it is illustrative and should not be mistaken for a verified inference from a specific input image.